

```
manchodu@Srini:~/OSLAB$ echo "Hello! This is input redirection." > input.txt
manchodu@Srini:~/OSLAB$ nano m5.c
manchodu@Srini:~/OSLAB$ gcc -o m5 m5.c
manchodu@Srini:~/OSLAB$ ./m5
```

```
=====
MEMORY MAPPED FILE I/O
=====
```

File size: 42 bytes

Data written using mmap().

Data read using mmap():
This file is being accessed using mmap().

Modified data using mmap():
Modified using memory mapped I/O.

Memory mapping completed successfully.

```
manchodu@Srini:~/OSLAB$ |
```

```
Memory mapping completed successfully.
```

```
manchodu@Srini:~/OSLAB$ nano m6.c
manchodu@Srini:~/OSLAB$ gcc -o m6 m6.c
manchodu@Srini:~/OSLAB$ ./m6
```

Data written using write().

Data read using read():
This file uses traditional read/write I/O.

Traditional read/write completed.

```
manchodu@Srini:~/OSLAB$ |
```

```
manchodu@srini:~/OSLAB$ cat m5.c
```

```
/* =====  
PART 2: mmap() FILE READING AND WRITING  
===== */  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <sys/mman.h>  
#include <sys/stat.h>  
#include <string.h>  
  
int main()  
{  
    int fd;  
    struct stat file_info;  
    char *mapped_file;  
    const char *filename = "mmap_file.txt";  
  
    printf("=====\n");  
    printf("      MEMORY MAPPED FILE I/O\n");  
    printf("=====\n\n");  
  
    /* Open file */  
    fd = open(filename, O_RDWR | O_CREAT, 0644);  
  
    if (fd == -1)  
    {  
        perror("open");  
        return 1;  
    }  
  
    /* Set initial file size */  
    const char *initial_data =  
        "This file is being accessed using mmap().\n";  
  
    size_t data_size = strlen(initial_data);  
  
    if (ftruncate(fd, data_size) == -1)  
    {  
        perror("ftruncate");  
        close(fd);  
    }  
}
```

```

{
    perror("ftruncate");
    close(fd);
    return 1;
}

/* Get file information */
if (fstat(fd, &file_info) == -1)
{
    perror("fstat");
    close(fd);
    return 1;
}

printf("File size: %ld bytes\n", file_info.st_size);

/* Map file into memory */
mapped_file = mmap(
    NULL,
    file_info.st_size,
    PROT_READ | PROT_WRITE,
    MAP_SHARED,
    fd,
    0
);

if (mapped_file == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return 1;
}

/* Write data through memory mapping */
memcpy(mapped_file, initial_data, data_size);

printf("\nData written using mmap().\n");

/* Read data through memory mapping */
printf("\nData read using mmap():\n");
printf("%s", mapped_file);

/* Modify the mapped memory */
const char *modified_data =
    "Modified using memory mapped I/O.\n";

memset(mapped_file, 0, file_info.st_size);

memcpy(
    mapped_file,
    modified_data,

```

```

        perror("mmap"),
        close(fd);
        return 1;
    }

    /* Write data through memory mapping */
    memcpy(mapped_file, initial_data, data_size);

    printf("\nData written using mmap().\n");

    /* Read data through memory mapping */
    printf("\nData read using mmap():\n");
    printf("%s", mapped_file);

    /* Modify the mapped memory */
    const char *modified_data =
        "Modified using memory mapped I/O.\n";

    memset(mapped_file, 0, file_info.st_size);

    memcpy(
        mapped_file,
        modified_data,
        strlen(modified_data)
    );

    /* Synchronize memory with file */
    if (msync(mapped_file,
        file_info.st_size,
        MS_SYNC) == -1)
    {
        perror("msync");
    }

    printf("\nModified data using mmap():\n");
    printf("%s", mapped_file);

    /* Unmap file */
    if (munmap(mapped_file, file_info.st_size) == -1)
    {
        perror("munmap");
    }

    close(fd);

    printf("\nMemory mapping completed successfully.\n");

    return 0;
}

```

```

fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0644);

if (fd == -1)
{
    perror("open");
    return 1;
}

/* Write using write() */
if (write(fd, data, strlen(data)) == -1)
{
    perror("write");
    close(fd);
    return 1;
}

printf("Data written using write().\n");

/* Move file offset to beginning */
lseek(fd, 0, SEEK_SET);

/* Read using read() */
ssize_t bytes_read =
    read(fd, buffer, BUFFER_SIZE - 1);

if (bytes_read == -1)
{
    perror("read");
    close(fd);
    return 1;
}

buffer[bytes_read] = '\0';

printf("\nData read using read():\n");
printf("%s", buffer);

close(fd);

printf("\nTraditional read/write completed.\n");

return 0;
}

```